# Análisis – EnvíaGT

## 2.- Requisitos funcionales y condiciones de validez

| Información conservada | Operaciones | Restricciones |
|---|---|---|
| Envíos: código, remitente, destinatario, peso, destino, estado e intentos | Cotizar y registrar | Código único y no vacío; peso positivo; cotizar no guarda ni cobra. |
| Datos particulares: dimensiones, valor declarado o temperatura | Calcular costo y días | Sobre ≤ 0.5 kg; dimensiones positivas; valor declarado positivo; temperatura −25 a 8 °C; refrigerado solo urbano. |
| Vehículos: placa, capacidad, refrigeración | Registrar y consultar | Placa única y no vacía; capacidad positiva. |
| Despachos: identificador, vehículo, envíos, estado y resultados pendientes | Crear, asignar, retirar, salir y registrar resultados | Un despacho abierto por vehículo; capacidad por peso real; compatibilidad; sin duplicar asignaciones; no salir vacío. |
| Ingresos: registro y reintentos, por categoría | Consultar reportes | No cobrar cotizaciones, asignaciones, retiros ni rechazos; Q15 solo al salir un reintento. |

## 3.- Reglas dependientes de la categoría

| Decisión | Responsable | Justificación |
|---|---|---|
| Costo base, peso facturable y recargos | calculateCost() de cada subclase | Despacho y sistema no conocen las fórmulas concretas. |
| Días hábiles estimados | getEstimatedDays() de cada subclase | El comportamiento cambia sin alterar el menú. |
| Límite de intentos | getMaxAttempts() de cada subclase | Shipment controla la transición y consulta la política polimórfica. |
| Refrigeración/ compatibilidad | requiresRefrigeration() e isCompatibleWith(Vehicle) | La clase de envío declara su requisito; Dispatch |

| | | |
|---|---|---|
| | | consulta el contrato. |
| Recargo por segundo intento en adelante | Dispatch.depart() y ShippingSystem.depart() | El despacho conoce la salida; el sistema centraliza el ingreso. |

# 4.- Fórmulas y estimaciones

| Categoría | Costo | Días urbano / departamental | Intentos |
|---|---|---|---|
| Envelope | Q20 / Q35 | 1 / 2 | 3 |
| Package | Base Q25 / Q40 + Q8 × ceil(max(peso real, largo×ancho×alto/5000)) | 2 / 4 | 3 |
| FragilePackage | super.calculateCost() + Q15 + 0.03 × valor declarado | 3 / 5 | 2 |
| RefrigeratedCargo | Q25 + Q12 × peso real + Q60 (solo urbano) | 1 / no permitido | 1 |

Interpretación: para la carga refrigerada se utiliza la tarifa base urbana Q25 indicada previamente para paquetes, porque el enunciado menciona «tarifa base según destino» sin definir otra tabla de tarifas para esta categoría. Esta interpretación queda explícita y es revisable con el docente.

# 5.- Estados y transiciones

| Objeto | Estado inicial | Operación | Estado final |
|---|---|---|---|
| Shipment | IN_WAREHOUSE | assign() | ASSIGNED |
| Shipment | ASSIGNED | unassign() | IN_WAREHOUSE |
| Shipment | ASSIGNED | dispatch() | IN_ROUTE |
| Shipment | IN_ROUTE | registerAttempt(true) | DELIVERED |
| Shipment | IN_ROUTE | registerAttempt(false), quedan intentos | IN_WAREHOUSE |
| Shipment | IN_ROUTE | registerAttempt(false), se agotaron intentos | RETURNED |
| Dispatch | PREPARING | depart() con al menos un envío | IN_ROUTE |

| | | | |
|---|---|---|---|
| Dispatch | IN_ROUTE | registrar último resultado pendiente | CLOSED |

DELIVERED y RETURNED son terminales. La reserva ASSIGNED evita dos despachos simultáneos. Un despacho en ruta no permite retirar ni agregar envíos; CLOSED libera el vehículo para un despacho nuevo.

# 6.- Responsabilidades, atributos y métodos definitivos

| Clase | Atributos (todos privados) | Métodos públicos principales / constructor |
|---|---|---|
| Shipment (abstract) | code: String; senderName: String; recipientName: String; weight: double; destinationType: DestinationType; status: ShipmentStatus; deliveryAttempts: int | Constructor protected(code, sender, recipient, weight, destination); getters; calculateCost(): double abstract; getEstimatedDays(): int abstract; getMaxAttempts(): int abstract; requiresRefrigeration(): boolean; isCompatibleWith(Vehicle): boolean; assign(), unassign(), registerAttempt(boolean): void; dispatch(): boolean; toString(): String. |
| Envelope | Sin atributos nuevos | Constructor público con campos comunes; sobrescribe costo, días e intentos. |
| Package | length, width, height: double | Constructor público con campos comunes y dimensiones; getters: double; getBillableWeight(): double; sobrescribe costo, días e intentos. |
| FragilePackage | declaredValue: double | Constructor público con |

| | | |
|---|---|---|
| | | campos de Package y valor declarado; getDeclaredValue(): double; sobrescribe costo, días e intentos. |
| RefrigeratedCargo | temperature: double | Constructor público con campos comunes y temperatura; getTemperature(): double; sobrescribe costo, días, intentos y refrigeración. |
| Vehicle | plate: String; capacity: double; refrigeration: boolean | Vehicle(String,double,boolean); getPlate(): String; getCapacity(): double; hasRefrigeration(): boolean; toString(): String. |
| Dispatch | id: int; vehicle: Vehicle; shipments: List<Shipment>; pending: Set<String>; status: DispatchStatus | Dispatch(int,Vehicle); getters; getLoadedWeight(), getAvailableCapacity(): double; addShipment(Shipment), removeShipment(Shipment): void; depart(): int; recordResult(String,boolean): void. |
| ShippingSystem | Mapas de envíos, vehículos y despachos; ingresos por categoría y concepto; acumulados; nextDispatchId: int | ShippingSystem(); quote(Shipment): double; registerShipment(Shipment), registerVehicle(Vehicle): void; createDispatch(String): Dispatch; assign(int,String), remove(int,String), depart(int), recordResult(int,String,boolean): void; getters de colecciones e ingresos; report(): String. |
| Main | input: Scanner static; | main(String[]): void; |

| | | |
|---|---|---|
| | system: ShippingSystem<br>static | métodos privados para menú, entradas y creación de envíos. |

Visibilidad: + público, − privado, # protegido. Los enums DestinationType, ShipmentStatus y DispatchStatus son públicos y contienen las constantes que aparecen en el UML. Los auxiliares privados y la firma exacta de cada constructor se consultan en el código y en el diagrama; las colecciones se exponen como vistas no modificables.

# 7.- Constructores, herencia y extensibilidad

Los constructores de Envelope, Package y RefrigeratedCargo llaman a super(...) para inicializar los atributos comunes y después validan sus datos específicos. FragilePackage llama a super(...) de Package para reutilizar dimensiones y validaciones. Su cálculo invoca super.calculateCost() y agrega embalaje y seguro; su tiempo invoca super.getEstimatedDays() y añade un día. Una alternativa de clases hermanas duplicaría el peso volumétrico y la tarifa, incrementando el mantenimiento. Para una nueva categoría se crea una subclase de Shipment que implemente costo, tiempo e intentos y, si procede, sobrescriba compatibilidad; se incorpora una opción de creación en Main. Dispatch no requiere cambios porque utiliza isCompatibleWith, getWeight y las transiciones de Shipment. Los reportes agrupan automáticamente por el nombre de la clase concreta, aunque podrían requerir cambios si se desea una etiqueta comercial distinta. Los estados no se modifican salvo que aparezcan nuevos procesos.